Week 4
2. Graphical User Interface Design
Usability Engineering

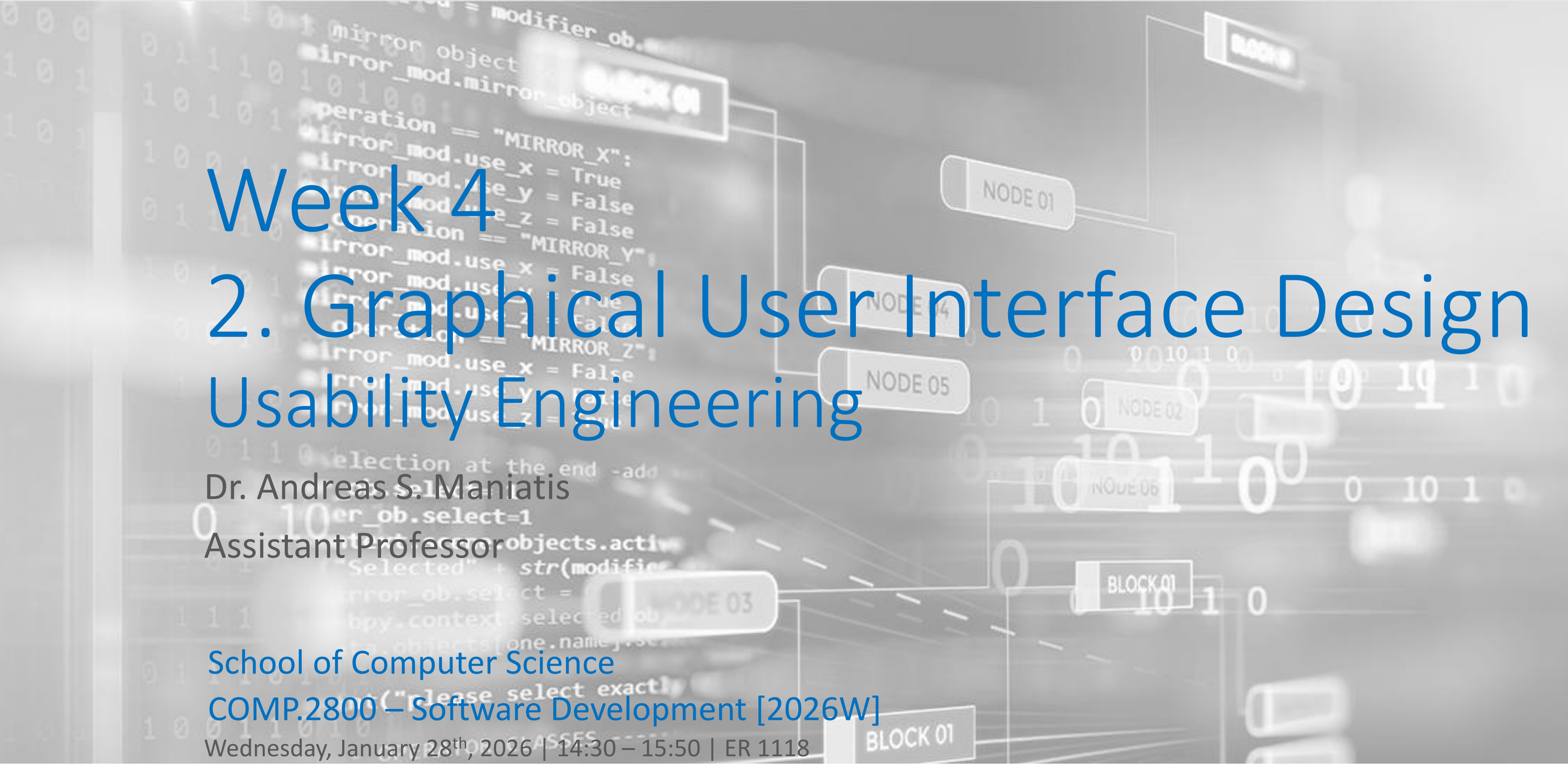
Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Wednesday, January 28th, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Agenda

Introduction to Usability Engineering

Usability Engineering Principles

Visual Cues

Gestalt Principles for Perceptual Organization

Graphical User Interfaces (GUI)

Hands On Example

Part I
Introduction to Usability Engineering

Introduction to Usability Engineering

University of Windsor

Usability Engineering

- Usability is a quality attribute
- “Usability” of an application refers to its
  - ease of use
  - ease of learning
  - appropriateness to the task being performed
- Usability can be increased through
  - early evaluation of the user interface
  - extensive and iterative requirements analysis
  - use of design guidelines
  - usability testing
  - user and task modeling
- Usability engineering integrates these activities

No Usability Engineering : The THERAC-25 example

A Malfunctioning Radiation Device

- Therac-25: new radiation device (circa mid-80's): updated an older, analogue device
  - made by Atomic Energy of Canada Ltd (AECL),
- Killed several people by radiating them with up to 70,000 times the allowable amount of radiation
- Company denied any fault but eventually an investigation discovered the problem (very infrequent)
  - Machine could be in two *modes*: electron and x-ray
  - The units were different in each mode
  - These modes could be confusing: operator could get in one mode, enter values, correct to other mode but only the screen would change while underlying mode remained the same ***if the correction was done too quickly***: result patient blasted with way too much radiation
  - Complicated by obtuse interface, terse error messages and failed observation cameras
- Extremely well studied case:
  - https://cs.union.edu/~fernandc/bio243/docs/THERAC-25.htm
  - https://tildesites.bowdoin.edu/~allen/courses/cs260/readings/therac.pdf

Therac-25: Root Causes

- Institutional causes:
  - AECL did not have the code independently reviewed.
  - AECL did not consider the design of the software during its reliability modelling.
  - The system documentation did not adequately explain error codes.
  - AECL personnel initially did not believe complaints.

- Technical causes (software and hardware):
  - The design did not have any hardware interlocks to prevent the electron-beam from operating in its high-energy mode without the target in place.

Therac-25: Root Causes

- Technical causes (continued):
  - The hardware provided no way for the software to verify that sensors were working correctly. The table-position system was the first implicated in Therac-25's failures; the manufacturer gave it redundant switches to cross-check their operation.
  - The equipment control task did not properly synchronize with the operator interface task, so that race conditions occurred if the operator changed the setup too quickly. This was evidently missed during testing, since it took some practice before operators were able to work quickly enough for the problem to occur.
  - The software set a flag variable by incrementing it. Occasionally an arithmetic overflow occurred, causing the software to bypass safety checks.
  - The software was written in assembly language. While this was more common at the time than it is today, assembly language is harder to debug than most high-level languages
- Similar cases:
  - Five worst user-interface disasters.
  - 20 Famous Software Disasters

Part II
Usability Engineering Disciplines

Usability Engineering : Disciplines

Usability Engineering Disciplines

- User interface design and evaluation is multidisciplinary
  - Graphic Designers
    - appearance of displays, icons.
  - UI Designers
    - interaction design.
  - Psychologists
    - understanding users' capabilities.
  - Sociologists
    - understanding work context in which application will be deployed.
  - Software Engineers
    - software design and construction.
  - Domain Specialists
    - E.g., doctors for medical systems, teachers for educational systems, etc.

Examples of Usability Problems

- System is poorly documented
- System is hard to use
  - Web banking
- System is hard to learn
  - voice mail
- System doesn’t match needs of users
  - lack of security on personal computers vs needs of University laboratories
- System is buggy or poorly supported
  - positioning figures in LaTex
- System is not trusted
  - e-commerce systems

User Interface Design Guidelines

1. Visibility of System Status
2. System Match to the Real World
3. User Control and Freedom
4. Consistency and Standards
5. Error Prevention
6. Recognition rather than Recall
7. Flexibility and Efficiency of Use
8. Aesthetic and Minimalist Design
9. Help Users Recognize, Diagnose and Recover from Errors
10. Help and documentation

References:

- User Interface Design Guidelines: 10 Rules of Thumb | IxDF
- 10 Usability Heuristics for User Interface Design - NN/G

Minimize Memorization

- Don't require the user to
  - memorize complex commands
- Make commands visible
  - through menus, icons, palettes
- Don't require the user to
  - memorize information from one screen to the next
- Find ways of reminding the user
  - about forgotten functionality

[From: George Miller, The Psychology of Communication, Basic Books, 1967]

Optimize Operations

- Frequently performed operations should be
  - simple, quick, error-free
- Interaction techniques
  - keyboard accelerators
  - macros
- User interface
  - adapt for variety of users, most importantly for expert users

Engineer for Errors

- Tendency is to assume correct usage is the norm
  - should treat erroneous use as the norm
- Application should be designed so that mistakes are not costly
  - undo operation
  - cancel operation
- Errors may be one-time or recurring
- Remember the Therac-25

Avoid “Modes”

- “Modes” cause an interpretation of input depending on context
- Danger is
  - user won’t understand what context they are in
  - user will forget what context they are in
  - cognitive burden of thinking about
    - “what does ‘k’ key mean now?”
- Modes can be acceptable
  - if they are visible and intuitive
  - Remember the Therac-25 (again!)

Be Consistent

- Consistency allows users
  - to reuse knowledge across and within applications
- Failure to be consistent
  - can lead to errors, frustration
- Capture corporate or platform style
  - Use consistent date format
  - Use consistent screen layout
  - Use same navigation commands from screen to screen
- Use consistent colour
- Use consistent dialogues
  - printing a file
  - saving a file
  - opening a file

Capabilities and Limitations of Humans

- Sight
  - most common way of receiving input from the computer – we see things on the screen
  - good at seeing movement
  - color, brightness, size, and depth
- Sound
  - tends to be under-utilized in interfaces but easy to abuse (can be distracting in offices for example)
  - provides strong feedback that an action has occurred
  - usually used in warnings or indications a process is about to start or end
  - cannot be relied upon (users mute sound or have poor speakers)

Capabilities and Limitations of Humans (cont.)

- Touch
  - not used in conventional interfaces, going to be the future, and already there are many!
- Memory
  - how much can a user remember? How long?
  - an average adult can remember 7 +/- 2 chunks of information at a time
  - if someone is distracted, the information can be lost
- Other characteristics of humans that may limit design
  - long term memory, problem solving abilities, motor control, emotions, and aging

Part III
Visual Cues

Visual Cues: Affordances

Visual Cues - Affordances

- Affordances
  - are the fundamental properties of the object that determine how it could possibly be used
  - Examples:
    - buttons are for pushing
    - knobs are for turning

Visual Cues – Affordances

University of Windsor

Visual Cues – Affordances

University of Windsor

Visual Cues – Affordances

What is wrong with this User Interface?

Not Intuitive

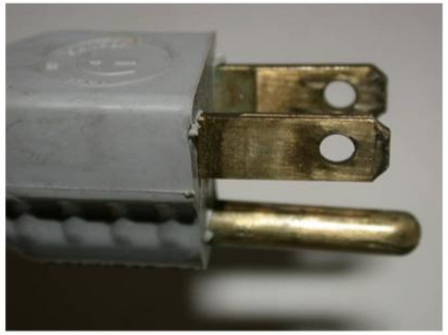
University of Windsor

Visual Cues – Affordances

- Affordances
  - are the fundamental properties of the object that determine how it could possibly be used
  - Examples:
    - buttons are for pushing
    - knobs are for turning
  - “What on this item helps me to figure out how I should use it?”
  - Need to be perceivable and intuitive
  - Examples on a computer:
    - buttons are clickable
    - can type in blank, white boxes

Visual Cues: Constraints

Visual Cues – Constraints

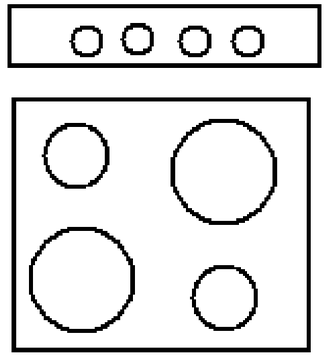


- Constraints
  - restrictions on affordances
    - size, proximity, type, or number
  - limit the uses that people will attempt
  - examples:
    - CD player has a tray that fits a CD perfectly
    - electrical outlet with one hole larger than the other
  - “What are the limitations of this characteristic/affordance which will help me figure out how to use the item?”
  - examples on a computer:
    - small textbox for 10 characters
    - vertical scroll bar along the window

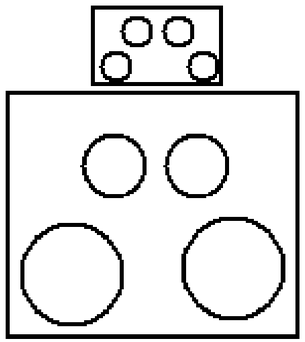
Visual Cues: Mappings

Visual Cues – Mappings

- Mappings
  - relationship between controls, movements, and results in the world
  - the position of controls in relation to the item they control
  - example
    - burners on a stove



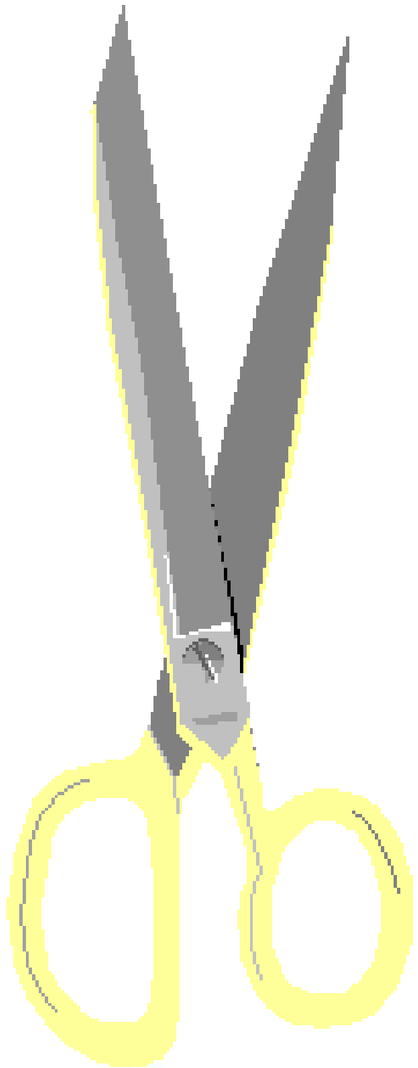
Which control for which burner?



Which control for which burner?

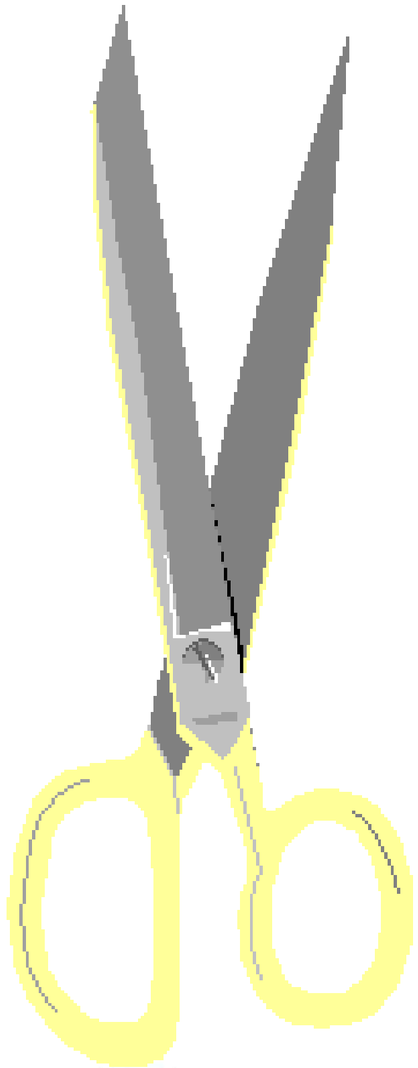
A Good Design Example

Example of a Good Design



Example of a Good Design

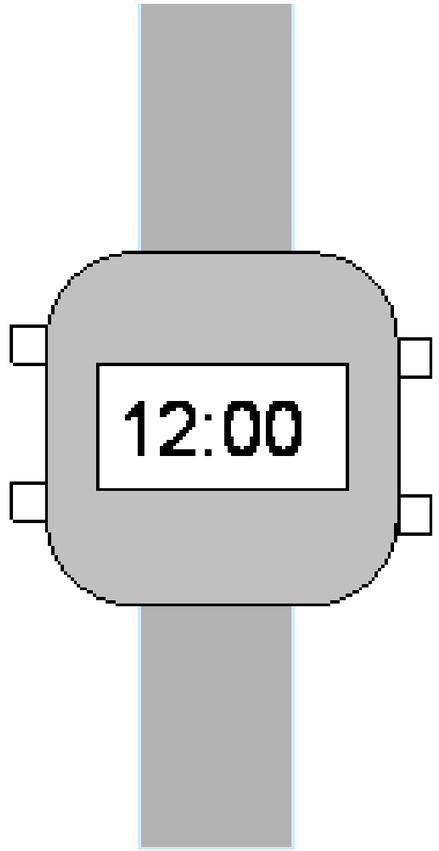
- Scissors
  - affordances
    - holes for something to be inserted
  - constraints
    - big hold for several fingers, small hole for thumb



A Bad Design Example

Example of a Bad Design

- Digital watch
  - affordances
    - four buttons to press
  - constraints and mappings
    - unknown: no visible relation between buttons, possible actions, and end results
    - time as shown is usually “too exact”



Part IV
The Gestalt Principles for Perceptual Organization

The Gestalt School of Psychology

Gestalt School of Psychology

- Members investigated how we group similar elements, recognize patterns and simplify complex images when we see objects in visual displays.
- Gestalt principles:
  - Pattern perception
  - Robust rules that describe the way in which we see patterns in visual displays
  - Actual mechanisms have not withstood the test of time (or science), but the principles have endured
  - There are many principles (at least 108!!)

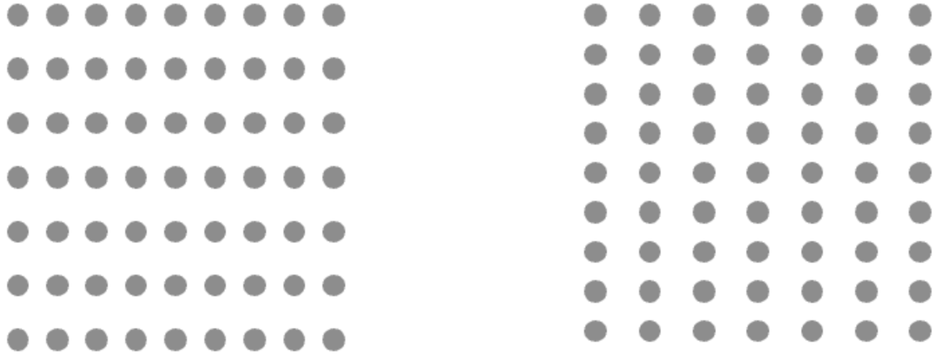
The Gestalt Principles

1. Proximity

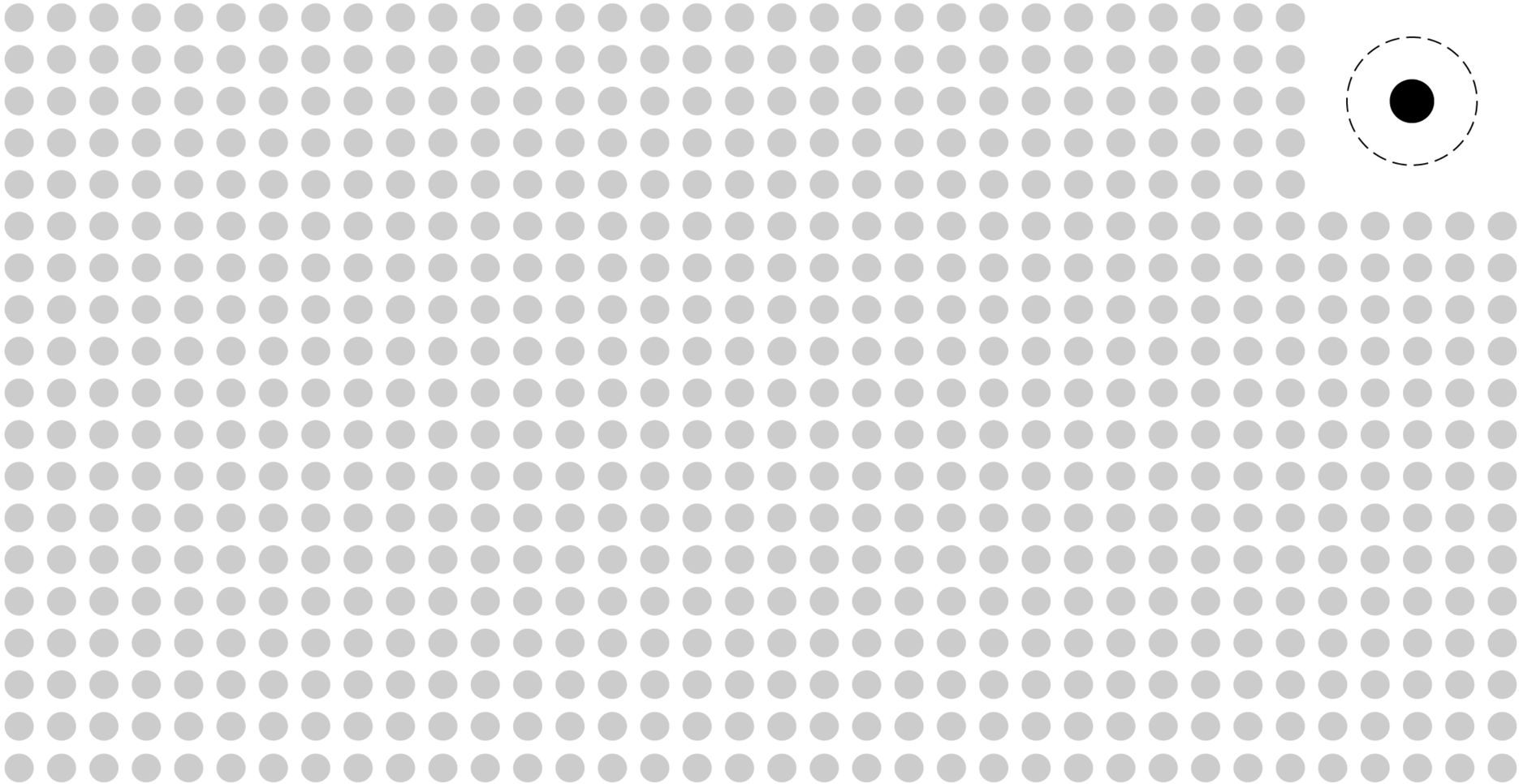
- Graphical elements that are close together are perceived as belonging together
  - Groups are perceptual units

Proximity

- A small change of spacing causes us to change what is perceived from a set of rows to a set of columns.

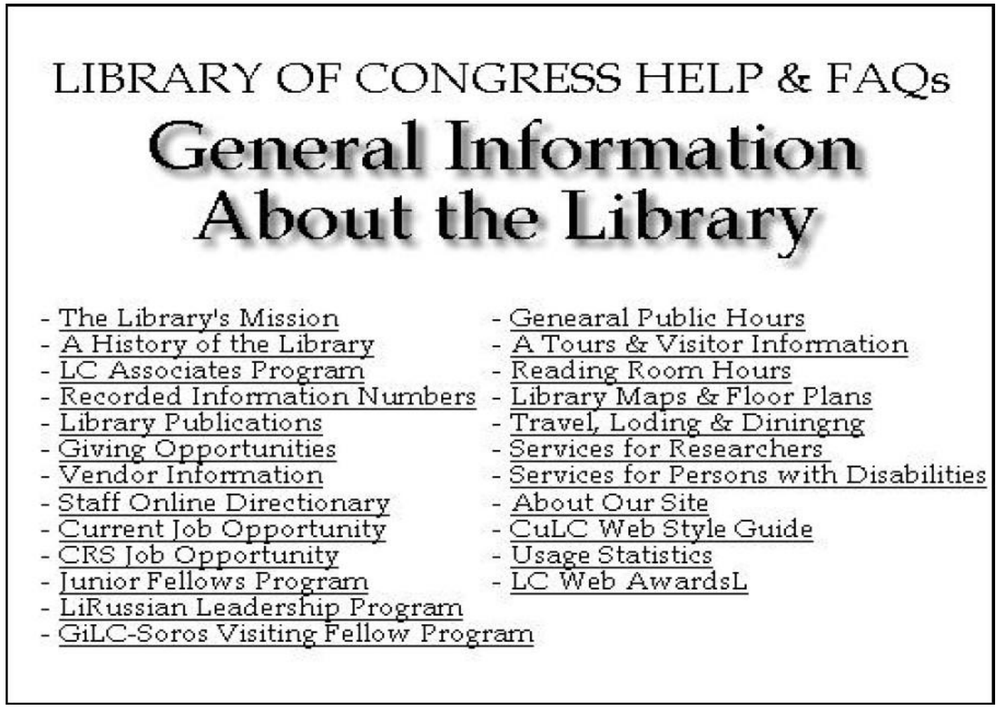


Outliers: Looking for the unexpected



Proximity

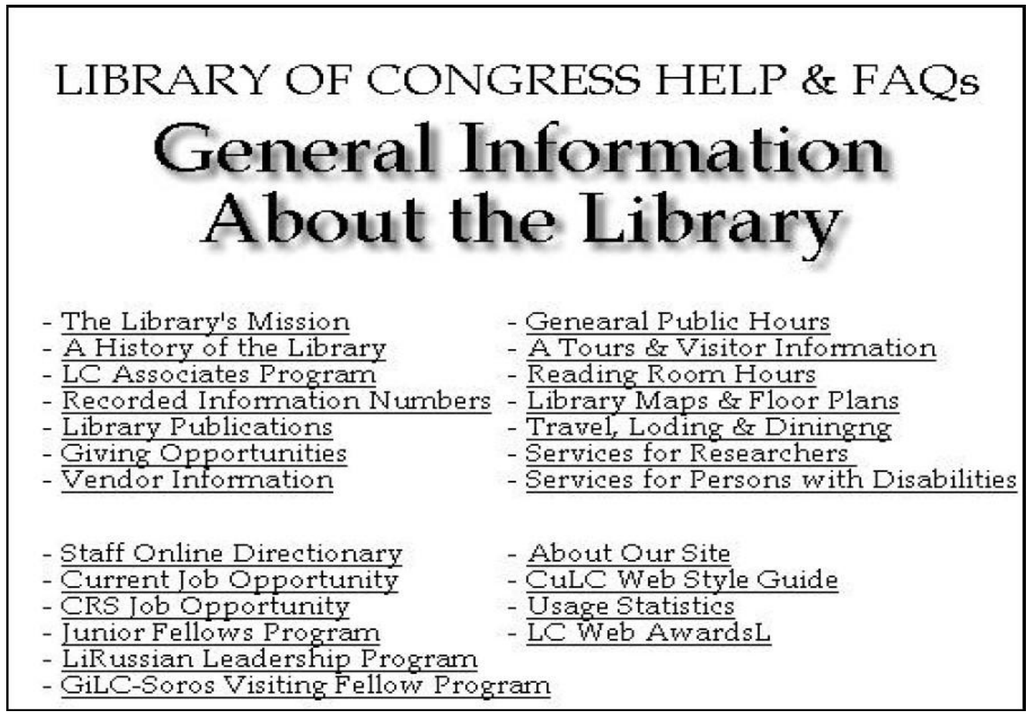
- It is hard to find things if they are not grouped into spatially organized clusters



51

Proximity

- Much easier to find things due to proximity in related content



University of Windsor

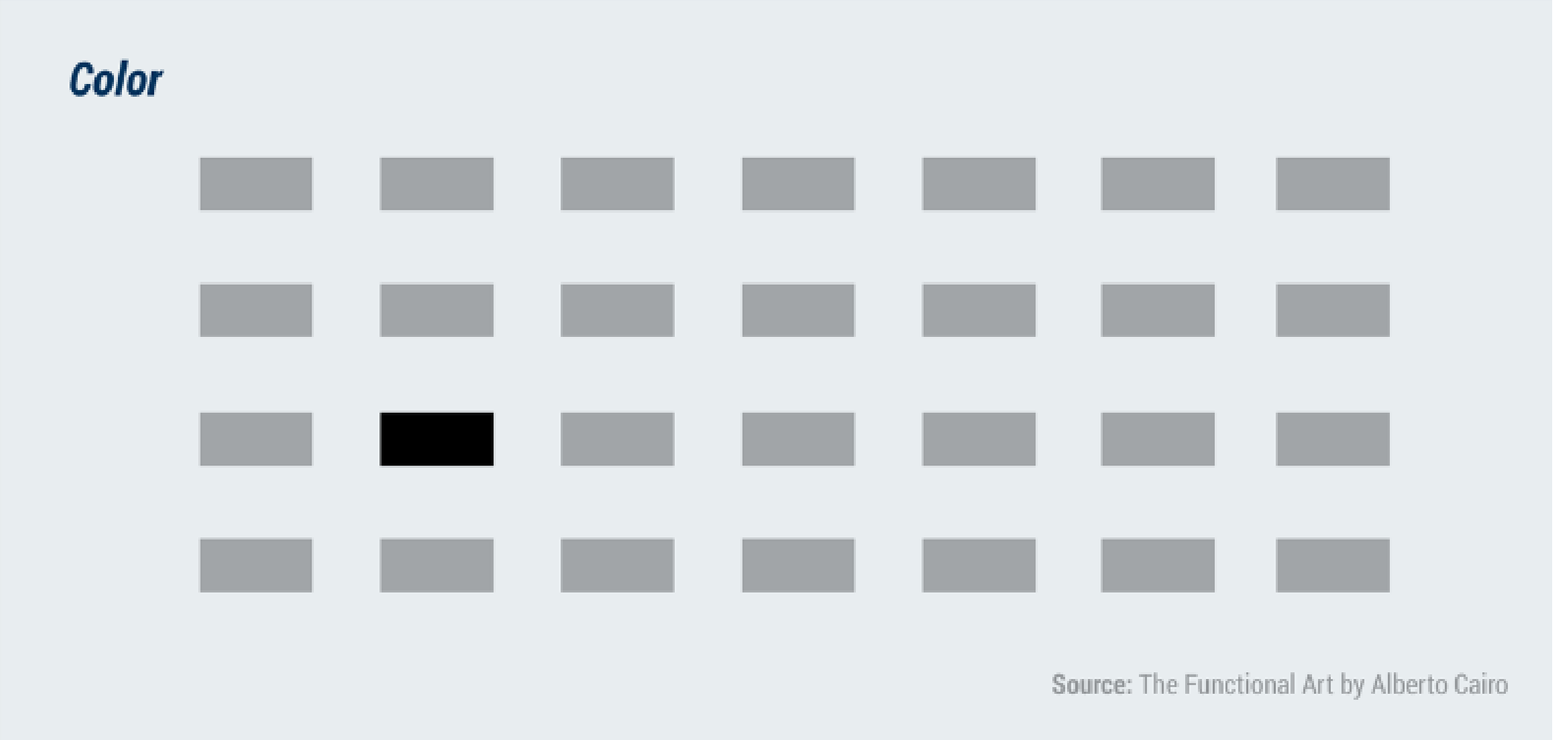
2. Similarity

Similar graphical elements tend to be grouped together

- Size
- Shape
- Colour
- Orientation
- Velocity (common fate)

Similarity between the elements in alternate rows causes the row perception to dominate

The Visual Brain (3) – Patterns

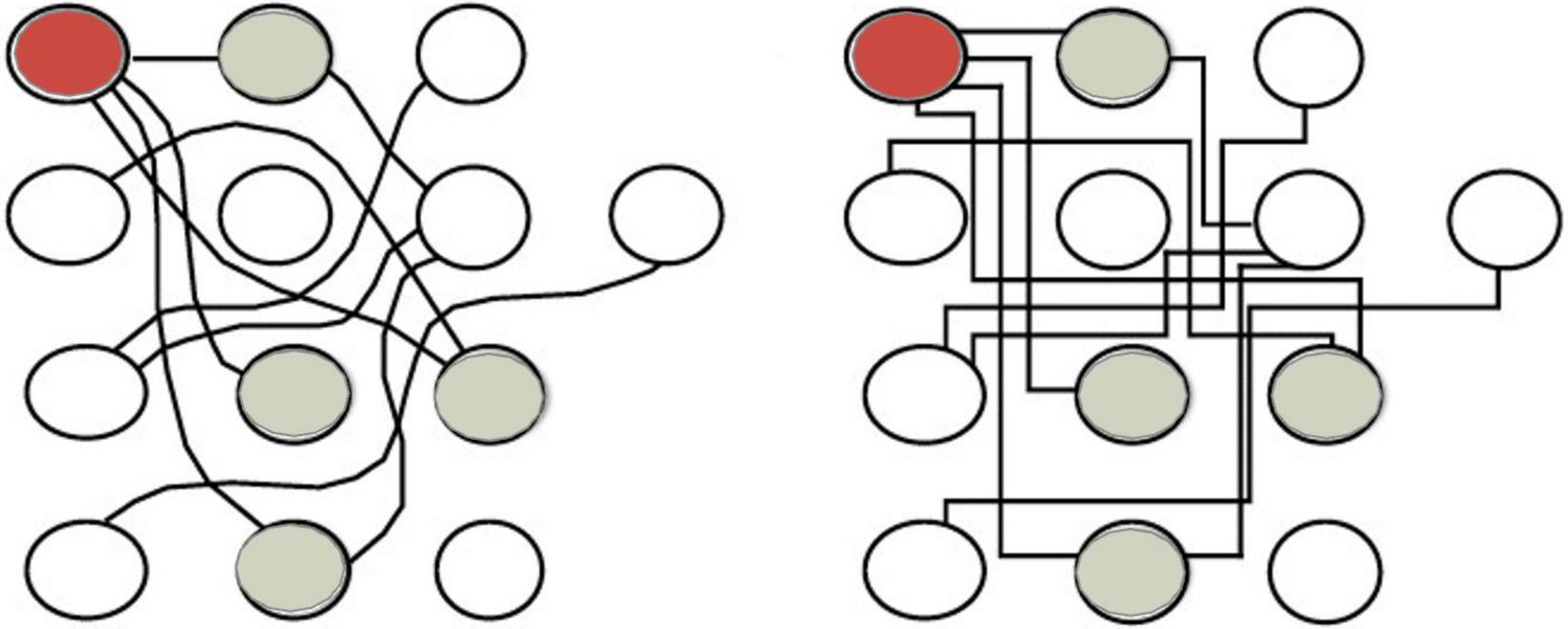


Source: The Functional Art by Alberto Cairo

3. Continuity

Graphical elements tend to be grouped together if along a path that is smooth and continuous, rather than with abrupt changes

Continuity

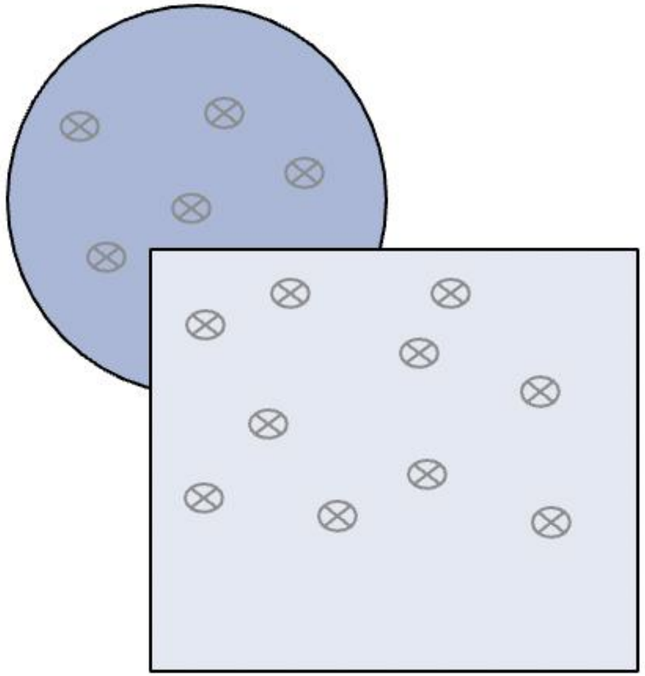


4. Common Region

Overlooked by Gestalt psychologists but added later

Graphical elements tend to be grouped together if contained within a common region

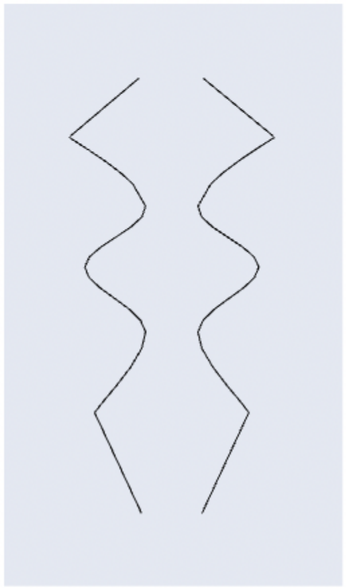
Closed contour tends to be seen as the boundary of an object

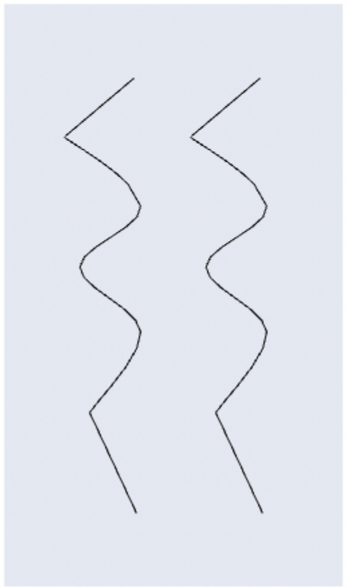


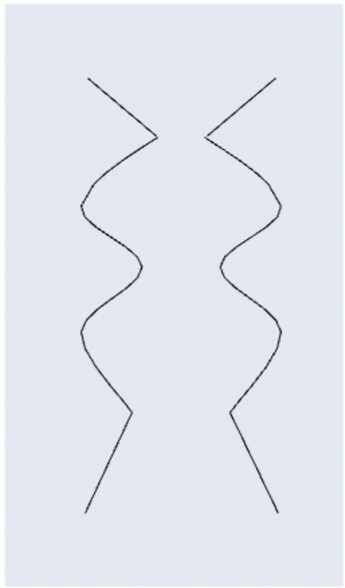
5. Symmetry

Two parts can form a visual whole when symmetry is used

Identical contours show how parallel lines look like two lines, whereas symmetrical lines look like a holistic figure

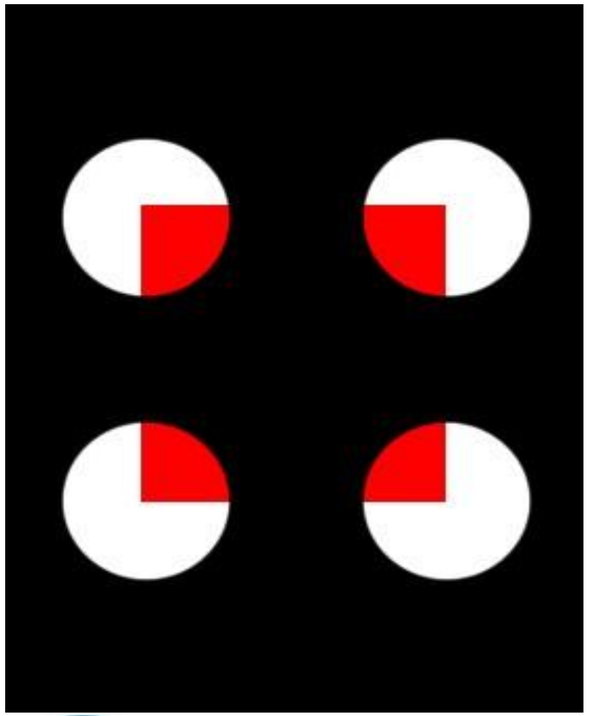






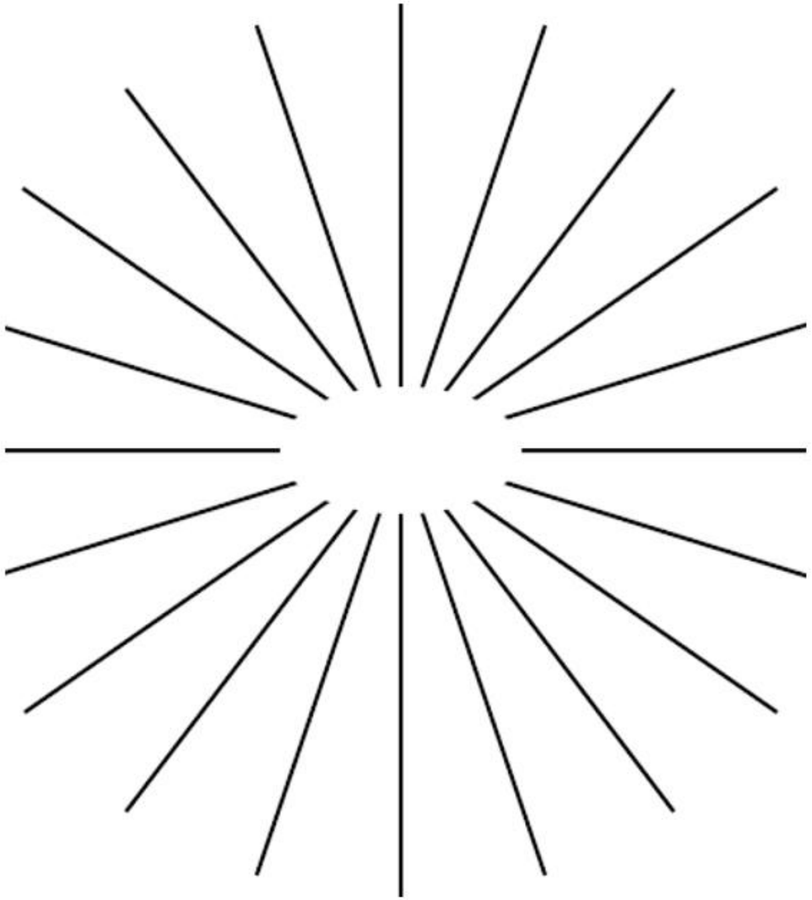
6. Closure

- A closed contour tends to be seen as an object
- If enough of the shape is indicated, people perceive the whole by filling in the missing information



The Visual Brain (4) – Difference

Closure



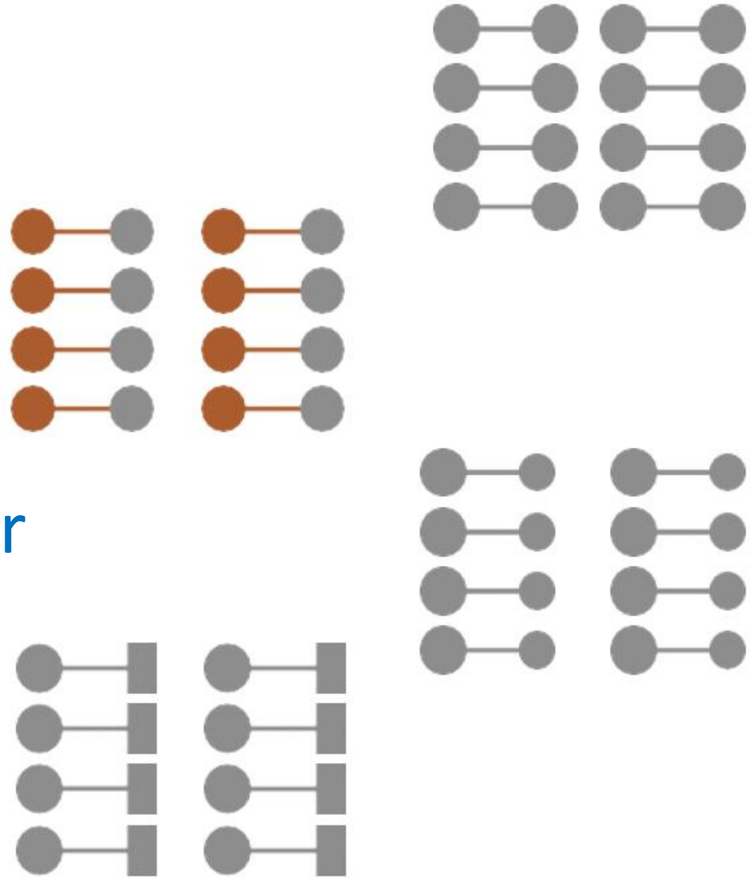
Illusory contours often results from this effect

They are the edges of an *inferred occluder*

- Note: a region surrounded by an illusory contour is often seen as somewhat brighter in colour

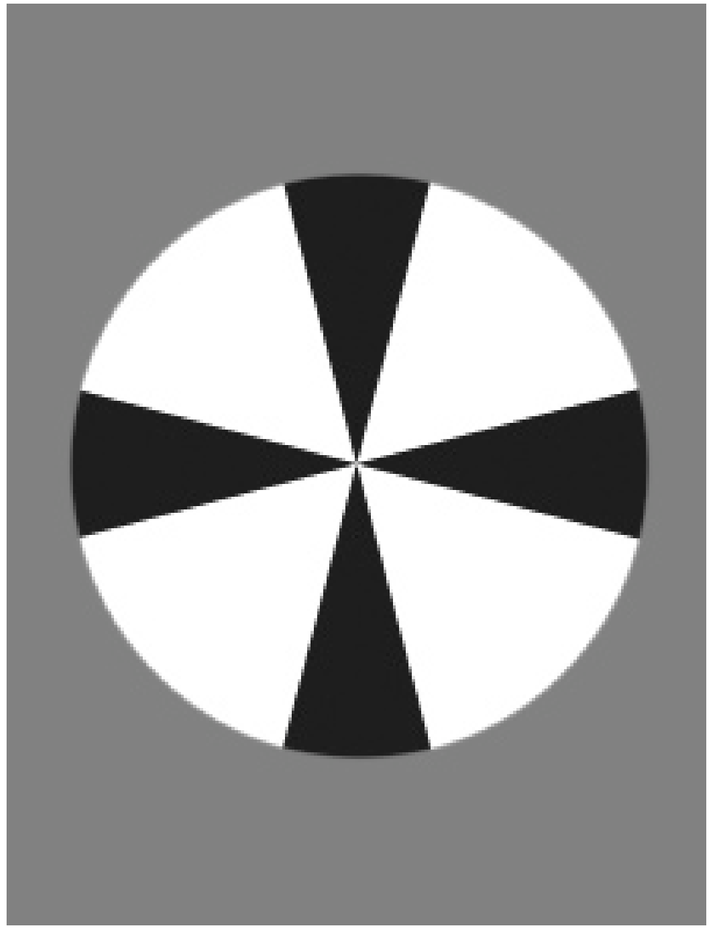
7. Connectedness

- Powerful tendency of the visual system to perceive any uniform, connected region as a single unit
- A powerful grouping principle that is stronger than proximity, color, size, and shape



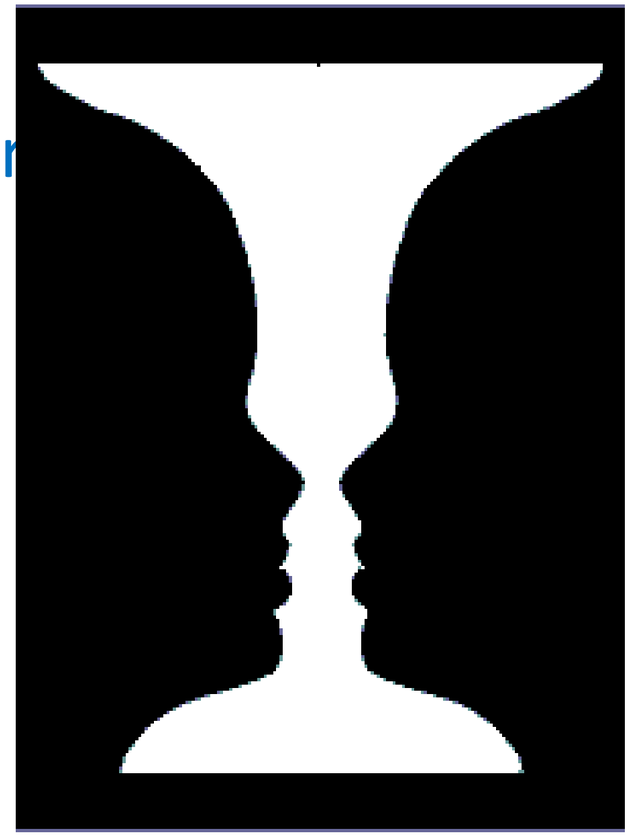
8. Relative Size

- Smaller components of a pattern tend to be perceived as objects
- E.g. a black propeller is seen on a white background instead of a white propeller on a black background



9. Figure/Ground

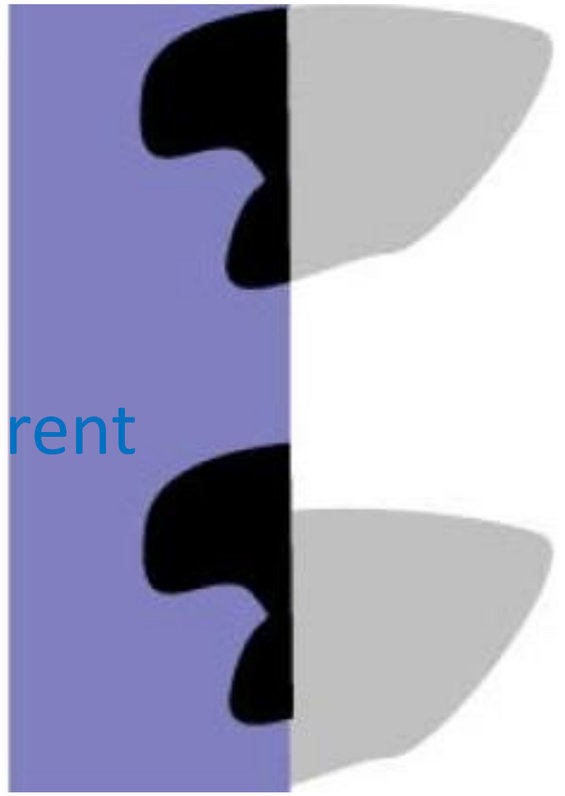
- A figure is something that is perceived as object-like in the foreground
- The ground is whatever behind lies behind the figur



- Rubin's Vase:
  - Can perceive either:
    - Two faces, nose to nose, or
    - A vase
  - The edge "belongs" to the figure

10. Perceptual Layering

- Effective perceptual layering allows the viewer to attend selectively to the elements in any one group with a minimum of interference from the others
- Layers should be processed selectively … it should be easy to ignore a layer that you aren't interested in
- E.g. continuity in regions, interpreted as semi-transparent surfaces



Part V
Graphical User Interfaces (GUIs)

How Visualization Works

Chart Challenge

FINANCIAL TIMES

The Chart Doctor

The science behind good charts

Take part in our interactive experiment to boost your chart-making confidence

References:
https://ig.ft.com/science-of-charts/
https://www.knowablemagazine.org/article/mind/2019/science-data-visualization

Chart Challenge

Click through the slide show to test your ability to decipher different types of graphs

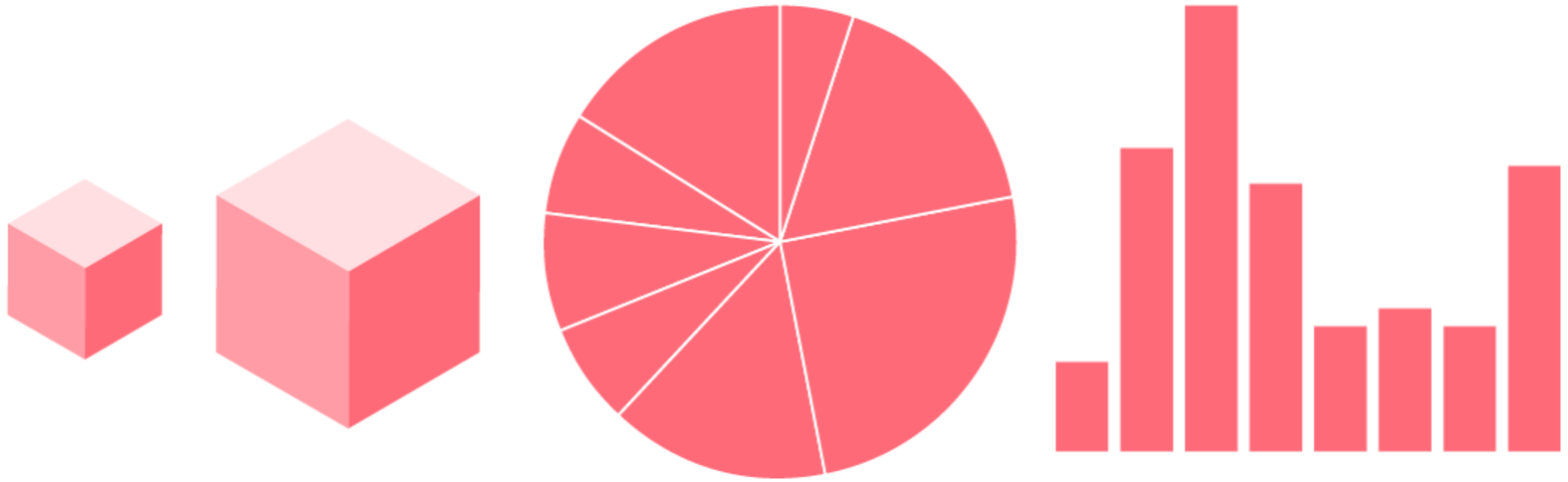
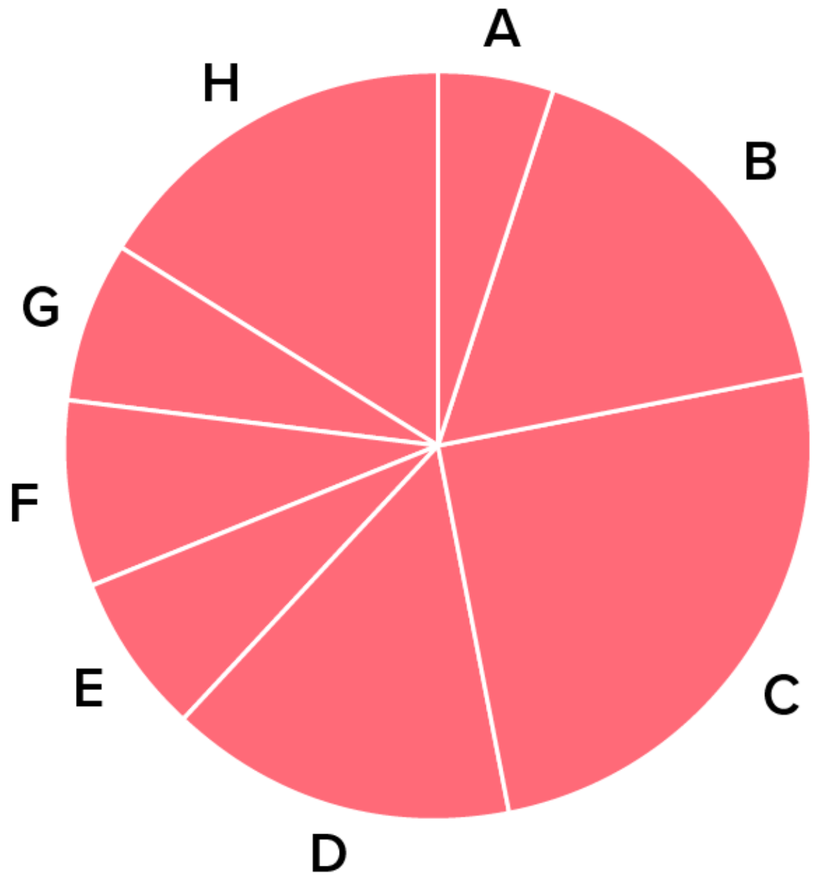


Chart Challenge

Seeing the wedges for the pie. Challenge 1 of 5



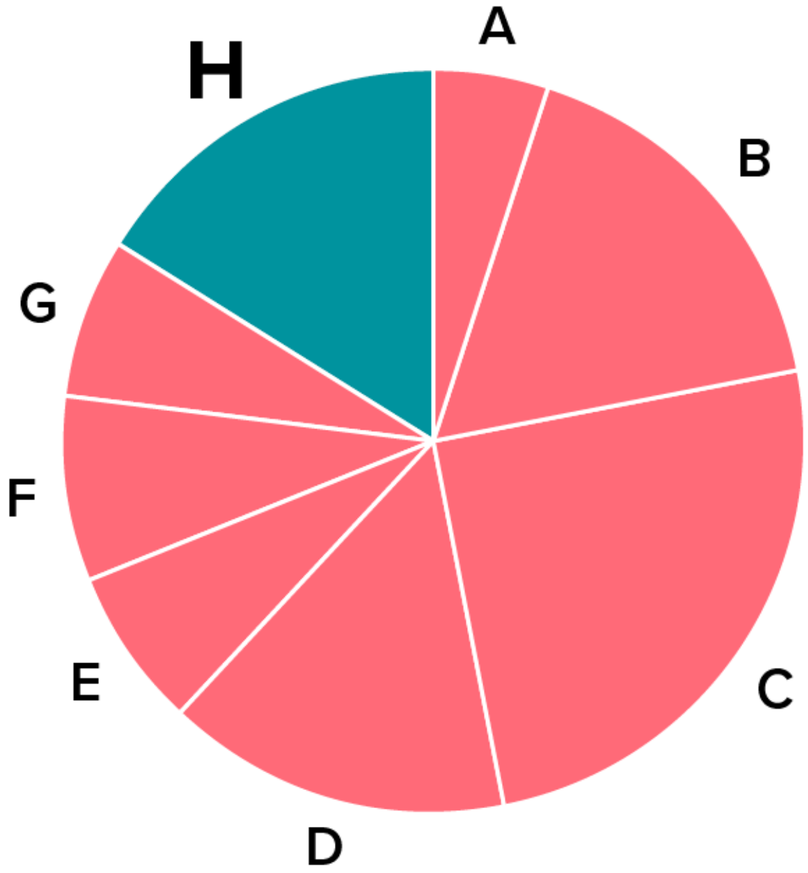
Which is the third largest segment in the pie chart?

- A
- H
- B
- D

University of Windsor

Chart Challenge

Challenge 1 Answer

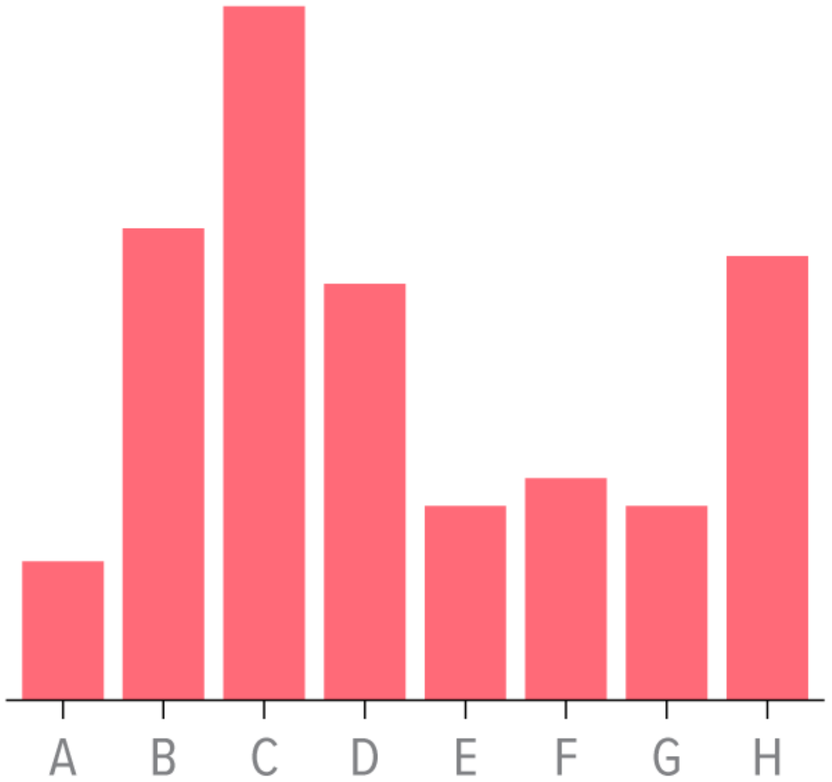


Which is the third largest segment in the pie chart?

- A
- H
- B
- D

Chart Challenge

Height anxiety. Challenge 2 of 5



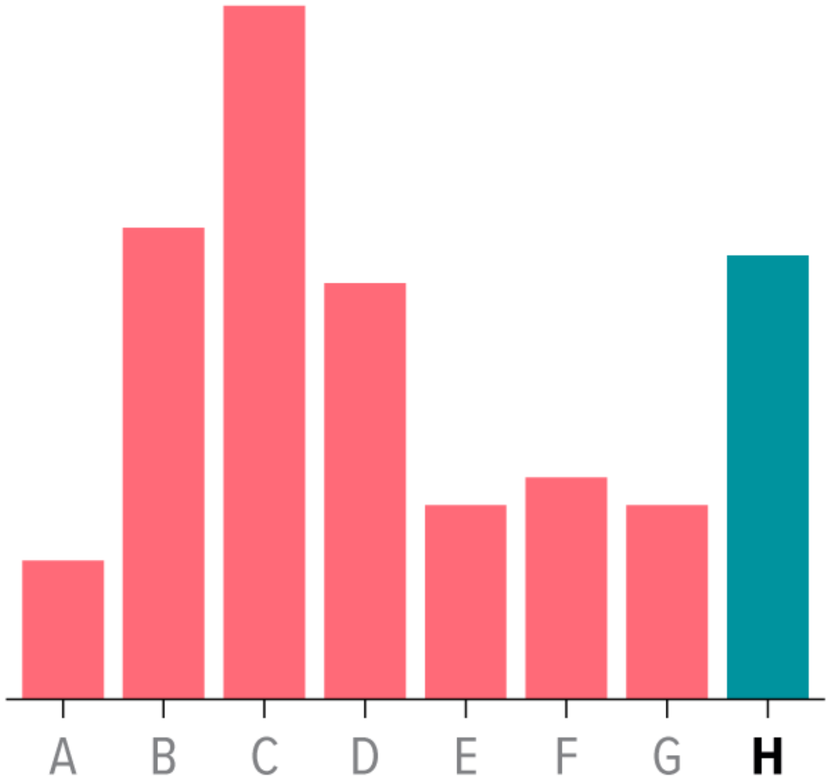
Which is the third tallest bar?

- H
- B
- D
- F

University of Windsor

Chart Challenge

Challenge 2 Answer



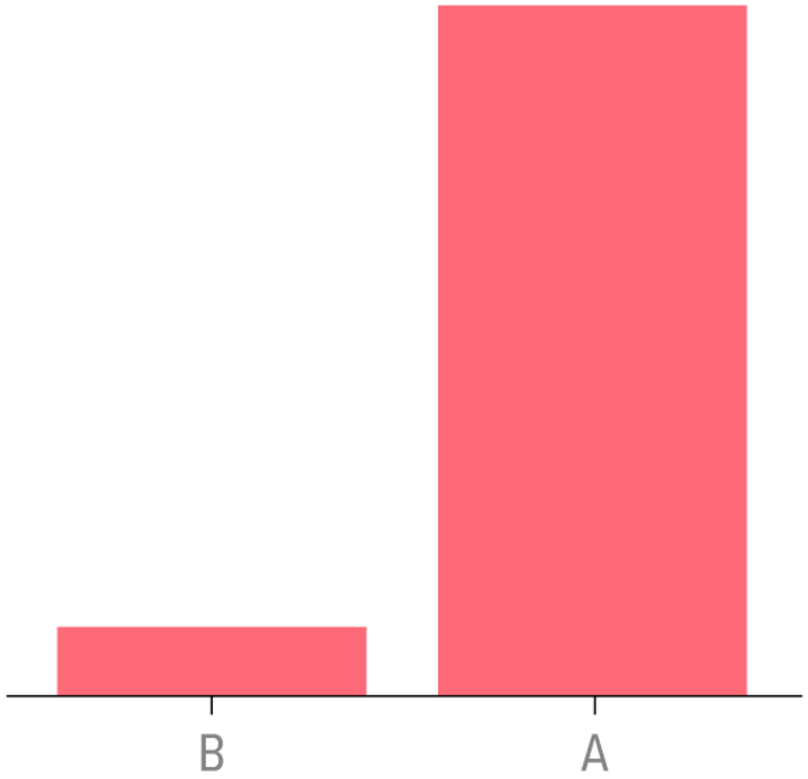
Which is the third tallest bar?

- **H**
- **B**
- **D**
- **F**

University of Windsor

Chart Challenge

Bar none. Challenge 3 of 5

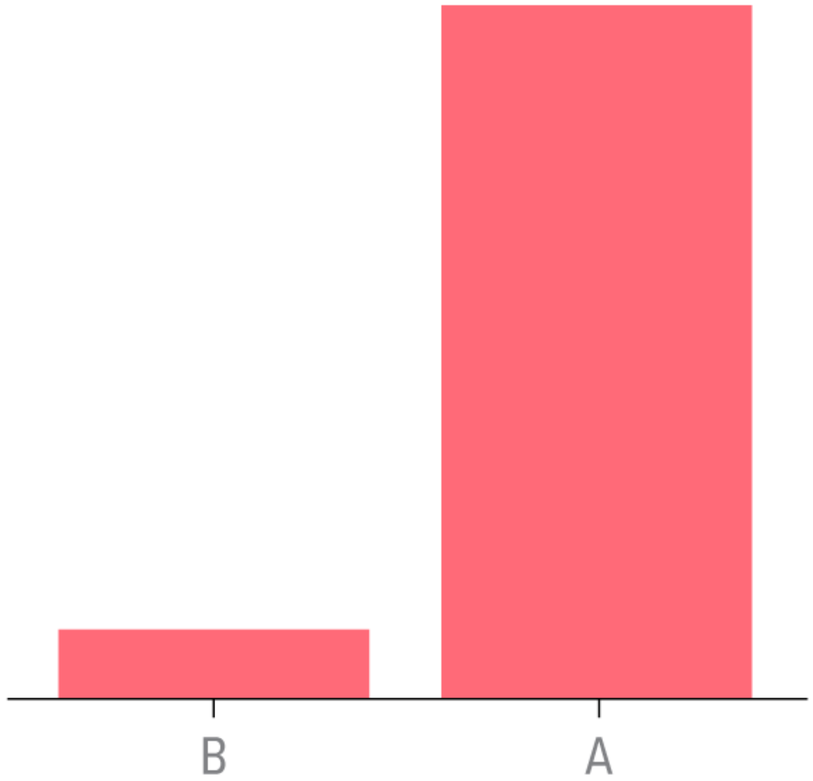


The value of A is how many times as large as the value of B?

- 9
- 8
- 10
- 5

Chart Challenge

Challenge 3 Answer

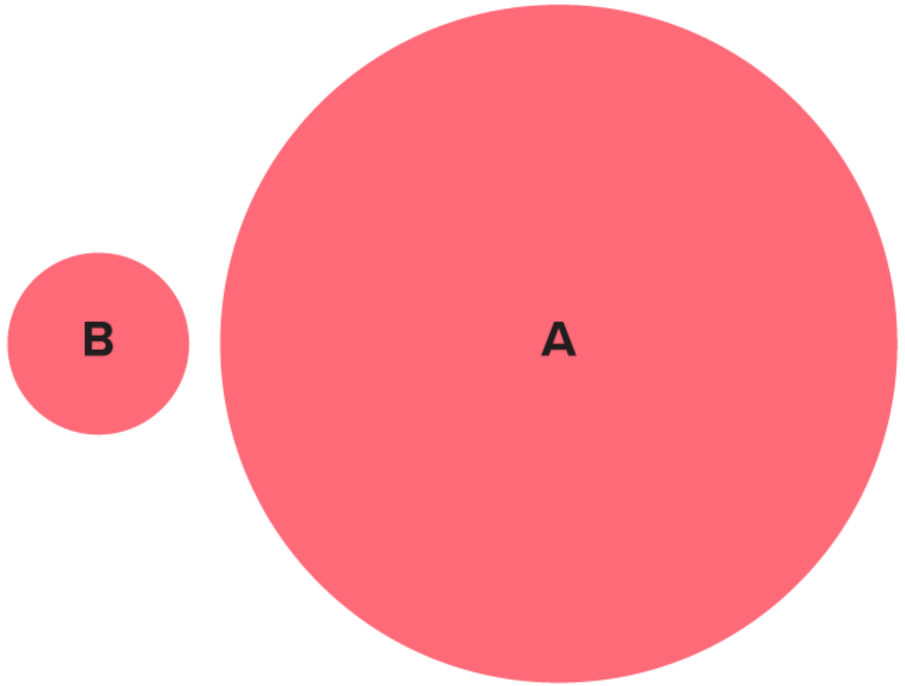


The value of A is how many times as large as the value of B?

- 9
- 8
- **10**
- 5

Chart Challenge

Eye on area. Challenge 4 of 5

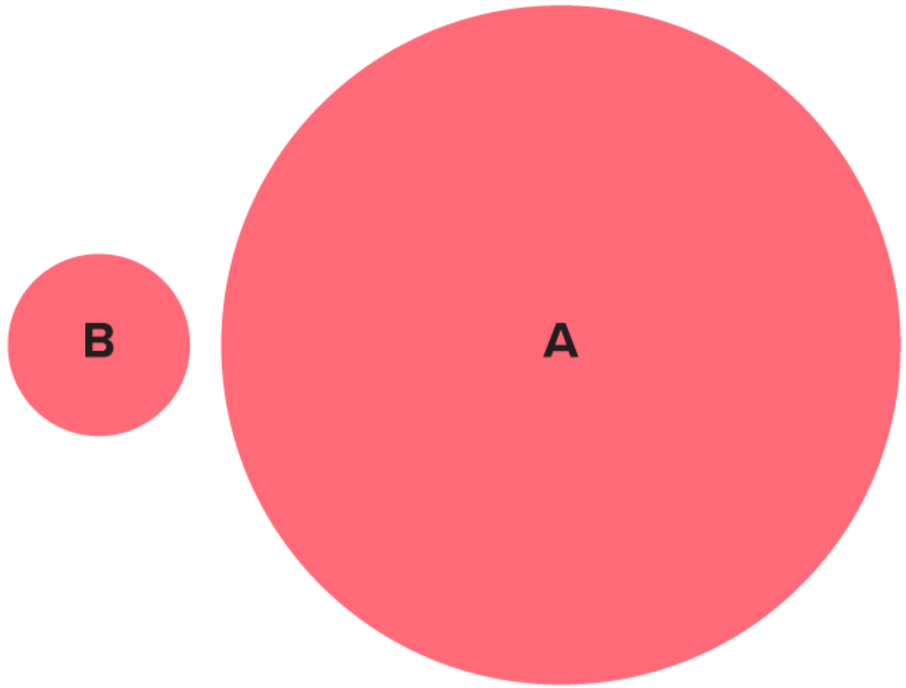


The area of circle A is how many times as large as the area of circle B?

- **4**
- **7**
- **10**
- **14**

Chart Challenge

Challenge 4 Answer

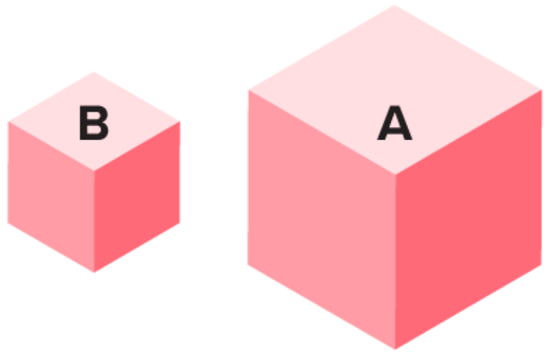


The area of circle A is how many times as large as the area of circle B?

- 4
- 7
- 10
- 14

Chart Challenge

Vast volumes. Challenge 5 of 5

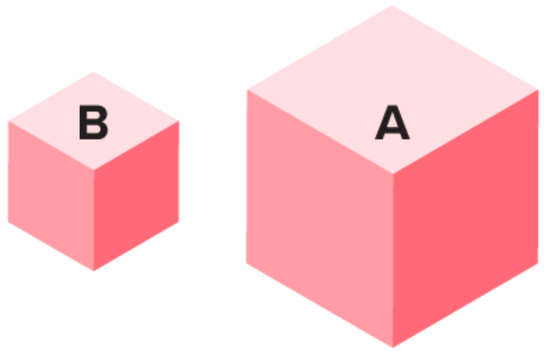


The volume of cube A is how many times as large as the volume of cube B?

- 3
- 5
- 7
- 8

Chart Challenge

Challenge 5 Answer



The volume of cube A is how many times as large as the volume of cube B?

- 3
- 5
- 7
- 8

The Power of Visual Analytics

12817368756138976546984506985604982867 82
98098584538224509856458945098450980965 65
90918302088059895957725888750506789045 67
88457898098216776548726649085609129496 86

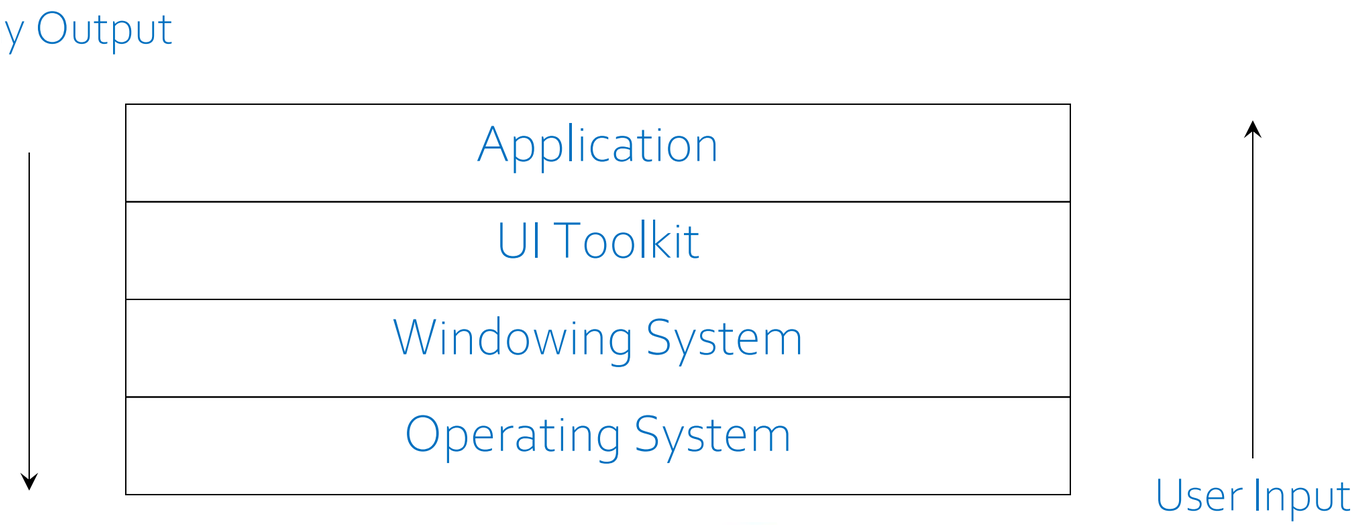
*12**8**1736**8**756138**9**765469**8**45069**8**56049**8**2**8**67**8**2 = 8*
*9**8**09**8**58453**8**2245098564589450984509**8**096585 = 8*
*9091**8**302088059**8**95957725**888**75050678904567 = 8*
*88457898098216776548726649085609129496**8**6 = 8*

32

How GUIs Work

How User Interfaces Work

- Several layers working together
- Input handled by the *event system*
  - a subset of each layer
  - GUI applications are *event-driven*



How User Interfaces Work – 2

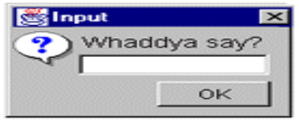
- Operating system
  - catches user input events and sends them to applications
- Windowing system
  - puts applications in separate windows
  - handles resize, close, and iconify events
  - enforces a *presentation style*
    - “look and feel” of title bars, menus, buttons, icons
- UI Toolkit
  - provides a set of high-level interface components (“widgets”)
    - e.g. menus, scrollbars, buttons, list boxes, etc.
  - in Java, choice of Abstract Windowing Toolkit (AWT) or JFC (Java Foundation Classes) Swing
  - provides basic support for layout and redrawing the screen

How User Interfaces Work – 3

- Application
  - decides what to do with user input
    - e.g. what does it mean to click on this button
  - manipulates the UI widgets
    - e.g. what to put in the list box, where to scroll to
  - controls repainting of the screen
    - all graphics must be handled by the application
- The software engineer (you) only has to deal with the application and toolkit layers

User Interfaces (UI) in Java

UI Widgets in Java



Entry

Buttons

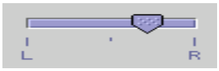
Label

Scroll pane

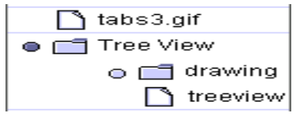
List

Table


Toolbar



Slider

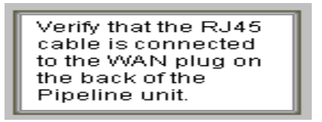


Tree view

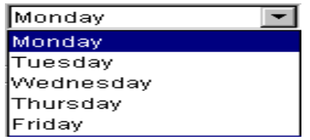

Progress Indicator

Tab pane

Text field



Text box



Combobox

- Use JFC Swing exclusively
- There are classes for each widget
  - in a class hierarchy

Reference: http://java.sun.com/docs/books/tutorial/uiswing/index.html

Input and Output

- Basic input
  - mouse motion, 2 or 3 buttons can be pressed
  - keys pressed on the keyboard
  - microphone (maybe)
- Basic output
  - graphics: text, pictures, animations, video
  - sound: beeps or audio files
- Beyond the basics
  - game controller, 3D position tracker, tablet, light pen, special sensors
  - 3D viewers, head-mounted displays, tactile output (force-feedback)
    - these require support at each UI level

Events

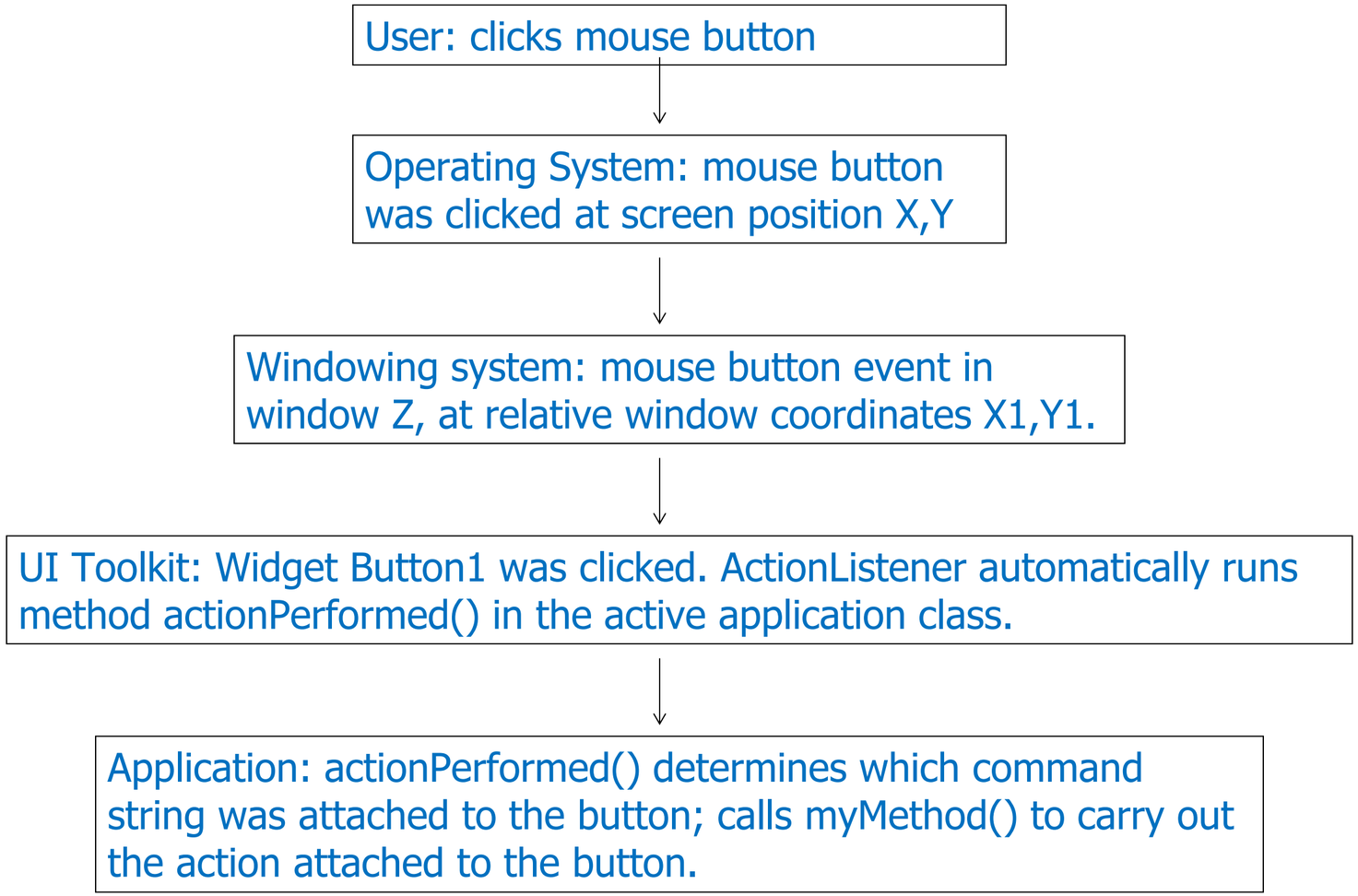
Events and Event Handlers

- User actions are events
  - e.g. button press, list scroll, draw line
  - note these are already high-level events!
- The application must respond to events
  - e.g. do the right thing when the “save” button is pressed
- How do we find out that events have occurred?
  - Java will automatically notify us, if we use
    - listeners (and adapters)
  - there are different listeners for different kinds of events
    - e.g. button listeners, mouse motion listeners
    - we must decide which classes should receive notice about the different events
- Once we have the event
  - determine what the event means
    - e.g. what button was pressed
  - call a method to carry out the appropriate action

Types of Events and Handlers

Act that results in the event	Listener type
User clicks a button, presses Return while typing in a text field, or chooses a menu item	ActionListener
User presses a mouse button while the cursor's over a component	MouseListener (MouseAdapter)
User moves the mouse over a component	MouseMotionListener (MouseMotionAdapter)
Table or list selection changes	ListSelectionListener
Component gets the keyboard focus	FocusListener
User closes a frame (main window)	WindowListener
Component becomes visible	ComponentListener

Life Cycle of a Mouse-Click Event

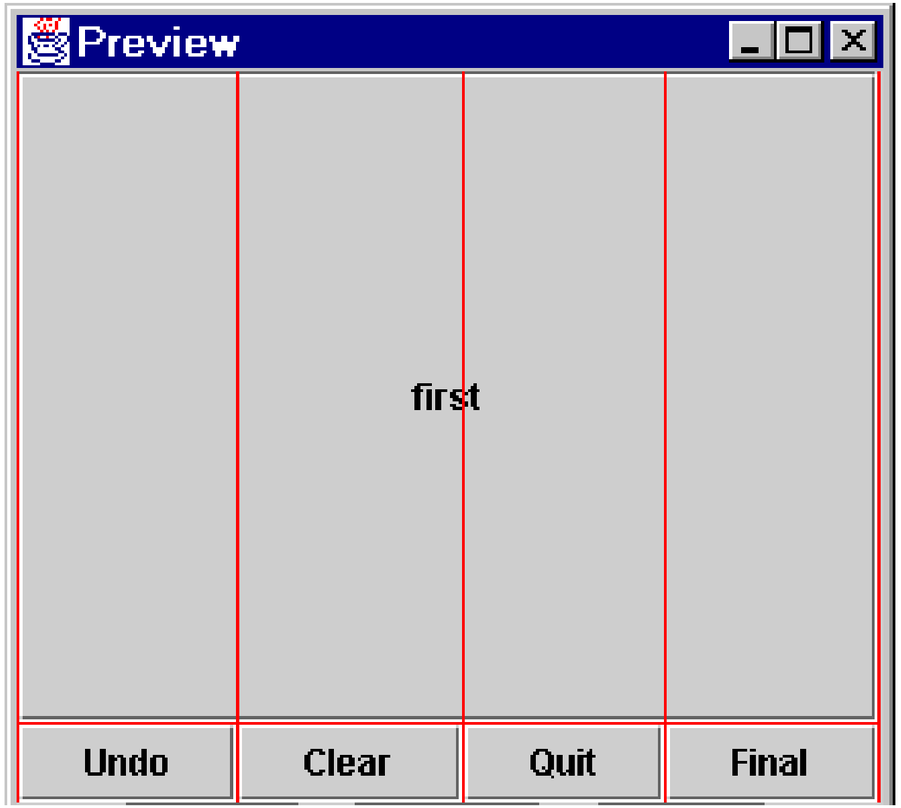


Painting

- Java widgets redraw themselves automatically
  - in programs that only use standard widgets, nothing to worry about
- What about custom graphics, animation, etc.?
  - must be redrawn by the program
  - software engineer must determine *when* and *how* to redraw the screen

Layout Managers

- How to get the widgets where we want them?
  - Use a layout manager
- Java has several layout managers
  - e.g. Flow, Border, GridBag
  - we will use GridBag exclusively (most powerful)
- GridBag Layout
  - puts widgets into rows in the window
  - forms a grid of cells
  - constraints are attached to each widget
  - constraints control various parameters:
    - how many grid cells are taken up
    - should the widget expand to fill its cells
    - who should expand when the window is resized
    - when to start a new row
  - see API for class GridBagLayout



Design Factors

- Visual organization
  - clusters of widgets or data
  - use of white space
  - alignment and spacing of elements
- Visual consistency
  - internal:
    - clusters are organized the same way
    - standard locations for standard buttons
  - external
    - follow style conventions for the platform

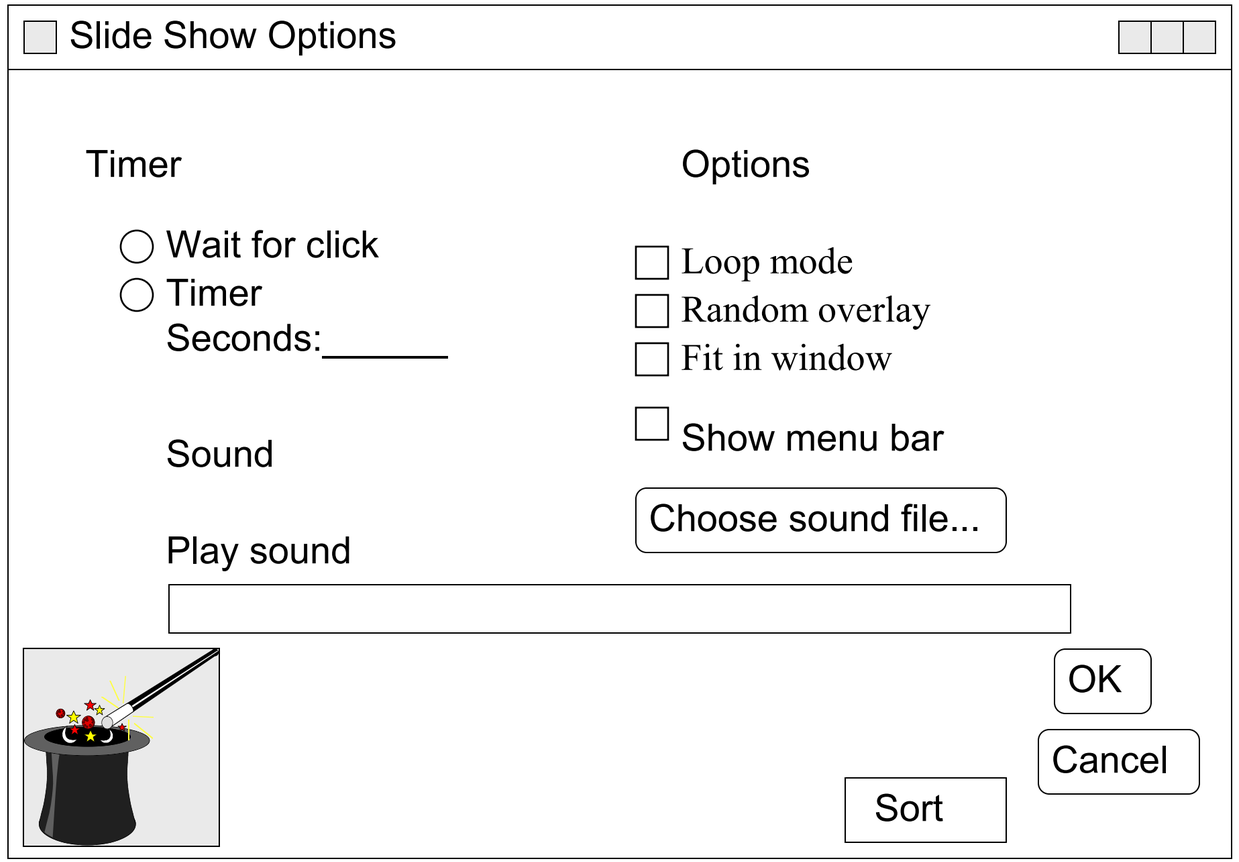
Design Factors – 2

- Legibility and readability
  - symbols and text should be clear & consistent
  - screen fonts
    - sans-serif & variable width (eg. Helvetica)
  - clear language
- Navigational cues
  - emphasize important elements
  - ordering should follow the task
  - flow should be top-left to bottom right

Part VI
Hands On Example

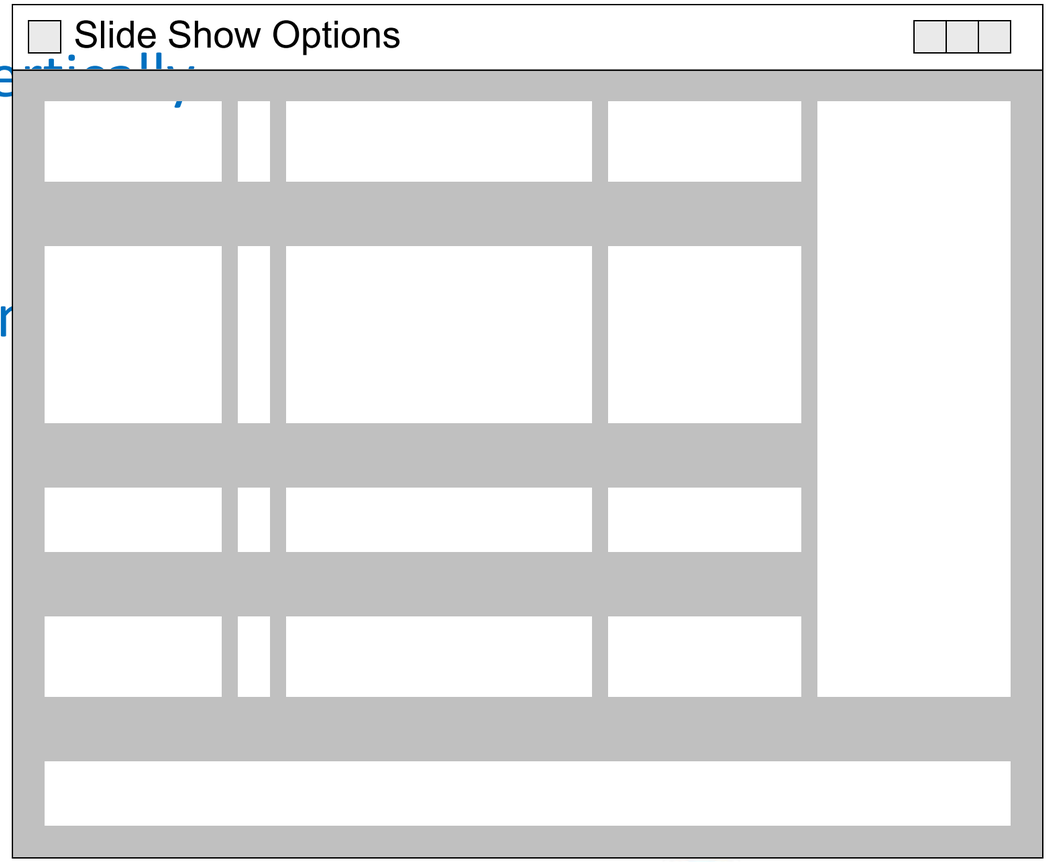
Screen Redesign Exercise

Screen Redesign Exercise



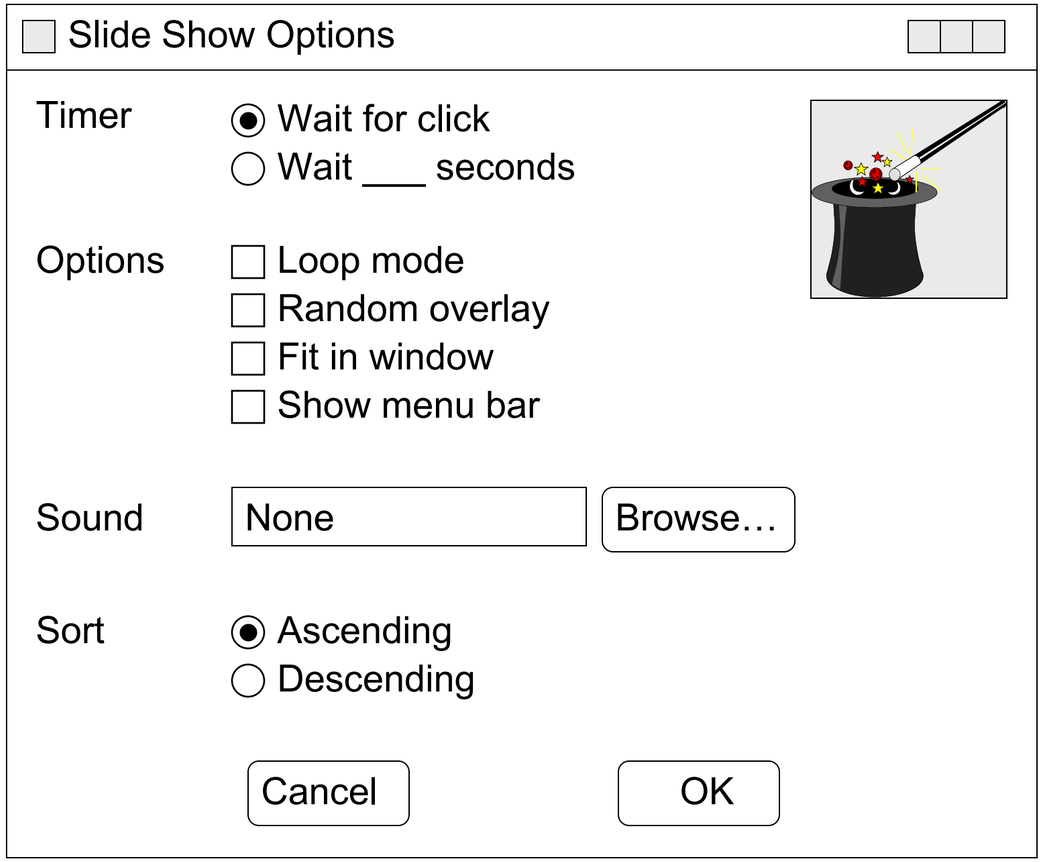
Redesigning the Slide Show Screen

- Use a grid system
  - groups of controls arranged ve
    - two-level hierarchy
  - specify fonts for each column
  - standard buttons at the bottor
  - move icon to top right



Slide Show Screen with Grid Overlay

Redesigned Slide Show Screen



Part VII
What's Next

Screen Redesign Exercise

Next lecture

- Next lecture includes:
  - Refactoring
  - Java Concurrency

Thank you!
Questions?

Dr. Andreas S. Maniatis
Assistant Professor
Andreas.Maniatis@UWindsor.ca
http://www.linkedin.com/in/andreasmaniatis

University of Windsor